

QLANKR · EVALUATION FRAMEWORK

How do you know your AI agent is good?

Measuring the quality of an AI-powered code review agent — from intuition to reproducible metrics.

← Prev

1 / 15

Next →

CONTEXT

What does the **agent** do?



Input

A GitHub Pull Request URL — any repo, any size.



8-stage AI pipeline

Reads code, calls knowledge graph tools, generates unit + integration or E2E test plans.



Output

Affected components, risk list, and ready-to-implement test specs.

The agent uses **human-in-the-loop checkpoints** — users review unit tests before choosing integration or E2E planning.

THE PROBLEM

Why "looks good" isn't enough



Non-deterministic — the same PR produces different outputs each run



No ground truth — there's no single "correct" test plan to compare against



Multi-step — a failure in stage 2 of 8 silently poisons all downstream output



Regressions are silent — a small prompt change can quietly drop quality weeks later

THE LANDSCAPE

Tools we considered

TOOL	TRACING	BUILT-IN EVALS	LANGGRAPH NATIVE	DATASET MGMT	SELF-HOST
LangSmith ✦	✓	✓	✓ native	✓	✗
Langfuse	✓	limited	manual	basic	✓
Weights & Biases	✓	limited	✗	✓	✗
Arize AI	✓	✓	✗	basic	✗

← Prev

4 / 15

Next →



Zero-config tracing

One environment variable. Every LangGraph run — every tool call, every LLM message — is automatically recorded.



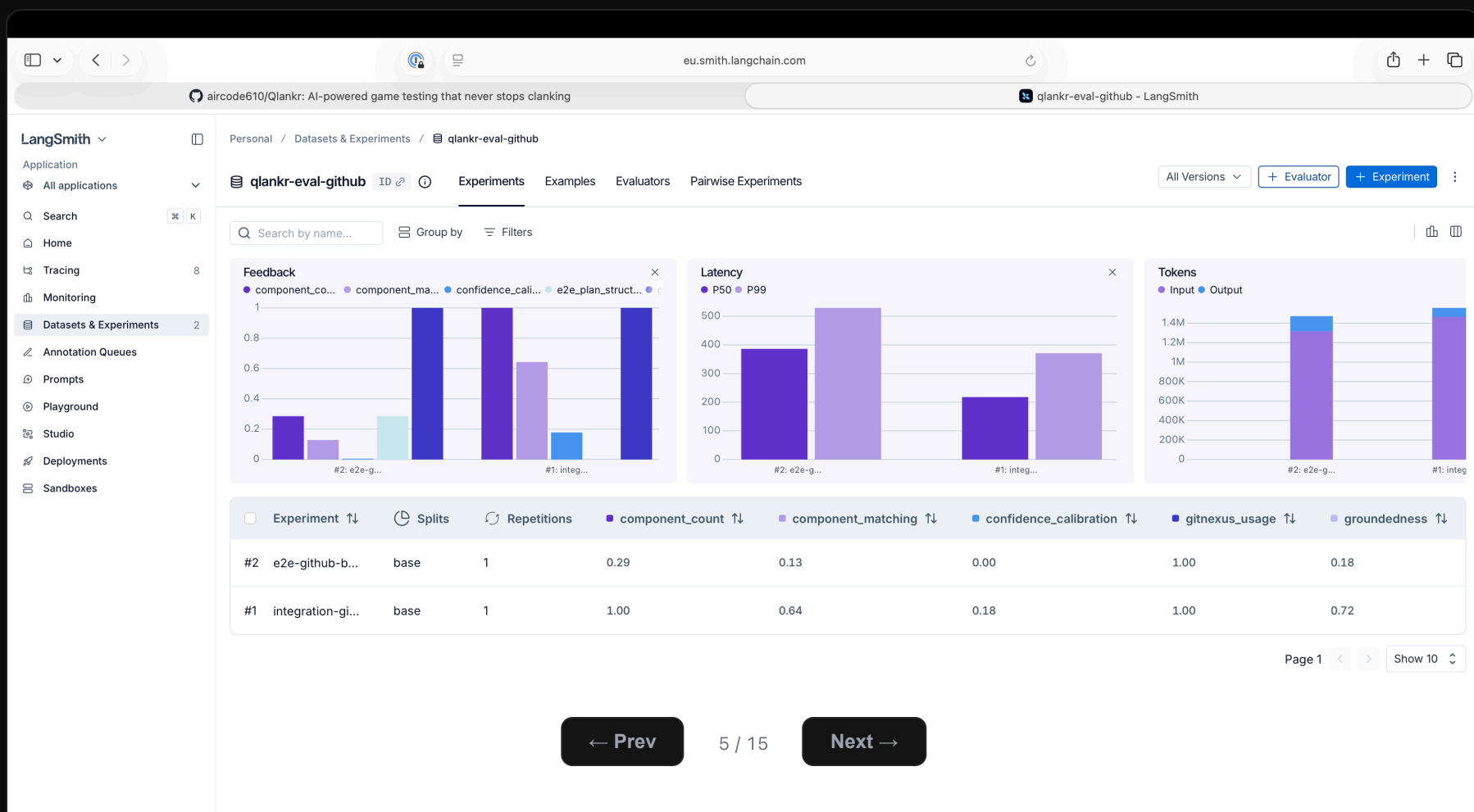
Datasets + Evaluators

Golden examples live in LangSmith.
Evaluators are plain Python functions — no framework to learn.



Experiment comparison

Change a prompt → re-run → see scores side-by-side across every metric, for every example.



GOLDEN DATASET

8 real PRs as ground truth



Qlankr repo — 1 PR

Our own codebase, fully indexed in the knowledge graph.

Tests the **full pipeline**: GitHub tools + graph queries (impact, cypher, context).



External repos — 7 PRs

OpenTTD, osu!, Cataclysm-DDA — the agent has never seen these repos.

Tests **GitHub-only fallback** across all PR sizes.

2

SMALL BUGFIXES

3

LARGE FEATURES

2

REFACTORS

1

NEW FILES ONLY

← Prev

6 / 15

Next →

EVALUATION DESIGN

15 evaluators across 3 layers

LAYER 1 — DETERMINISTIC

Pipeline health

Did the agent complete? Did all stages run? Are all required fields present?

Free to run — pure code, no LLM cost.

LAYER 2 — DETERMINISTIC

Behaviour checks

Did it use the right tools? Was it efficient? Did it use the knowledge graph when available?

Free to run — checks tool call lists.

LAYER 3 — LLM JUDGE

Quality judgements

Did it catch the right risks? Are test specs useful? Are claims grounded in real code?

Scored by Claude — 0.0 to 1.0 with reasoning.

Did the pipeline complete correctly?

BINARY

no_crash

"Did the agent finish without an error?"

Returns 1.0 if `affected_components` is non-empty in the output. Returns 0.0 if an error key is present. Returns 0.5 if it completed but found nothing.

RATIO

pipeline_progression

"Did the agent reach all expected stages?"

Checks `stages_seen` against the expected list for the path taken.
Integration path expects: `gather → unit_tests → integration_tests`
Score = stages hit / stages expected.

AVG

output_completeness

"Did every component have all 5 required fields filled?"

For each component: checks `component name` , `files_changed` , `impact_summary` , `risks` , `confidence` — scores filled/5, then averages across components.

RATIO

component_count

"Did the agent find enough components?"

Score = `min(1.0, found / expected_minimum)` . Each dataset example specifies a `min_components` threshold from human review.

Did the agent behave correctly?

RATIO	tool_coverage <i>"Did the agent call all the tools it should have?"</i> Score = <code> called n expected / expected </code>. Each dataset example lists the tools a thorough analysis requires (e.g. <code>get_file_contents</code>, <code>impact</code>, <code>submit_unit_tests</code>).
PENALTY	tool_efficiency <i>"Did the agent waste tool calls by repeating itself?"</i> Counts any tool called more than 4 times as "redundant". Score = <code>1 - (redundant / total)</code>. Submit tools are excluded from this check.
BINARY	gitnexus_usage <i>"Did the agent use the knowledge graph when the repo was indexed?"</i> Only applies when <code>expect_gitnexus=True</code> in the dataset. Checks that at least 2 of <code>impact</code>, <code>context</code>, <code>query</code>, <code>cypher</code> were called. Returns N/A (1.0) for GitHub-only repos.
RATIO	confidence_calibration <i>"Is the agent's confidence level realistic?"</i> For non-indexed repos: expects all components to have <code>confidence="low"</code> (no graph data). For indexed repos: expects at least half to be <code>medium</code> or <code>high</code>.

Did the agent produce **quality output**?

JUDGE

risk_quality

"Did the agent catch the risks a human reviewer identified?"

Claude is given the agent's risks and the golden expected risks. It scores recall (were the expected concerns found?), precision (were extra risks relevant?), and penalises missed critical ones. Exact wording doesn't matter — semantic match counts.

JUDGE

component_matching

"Did the agent name components the same way a human would?"

"ReAct Agent Core" should match "ReAct Agent". Exact string comparison would fail — Claude judges semantic equivalence between the agent's names and the golden names.

JUDGE

groundedness

"Are the agent's claims backed by real code changes?"

Claude is given the PR URL and the full analysis. It asks: "could a human reading the same PR arrive at these claims?" Penalises hallucinated file names, invented risks, or fabricated symbols.

JUDGE

unit_test_quality & integration_test_quality

"Are the generated test specs actually useful to a developer?"

Claude checks: Are targets specific real symbols? Do test cases cover edge cases? Are mocks appropriate? Are integration points real cross-module boundaries? Returns 0.0–1.0 with reasoning.

INTEGRATION-GITHUB · 7 EXTERNAL PRS

Our results

PIPELINE HEALTH

no_crash		1.00
pipeline_progression		1.00
output_completeness		1.00
component_count		1.00

BEHAVIOUR

tool_efficiency		0.89
tool_coverage		0.86

QUALITY (LLM JUDGES)

groundedness		0.72
risk_quality		0.66
component_matching		0.64
unit_test_structure		0.71
unit_test_quality		0.48
integration_test_structure		0.29
integration_test_quality		0.18
confidence_calibration		0.18

What the scores tell us



Pipeline is solid

no_crash, pipeline_progression, output_completeness, component_count all at 1.00. The agent reliably completes the full pipeline and finds at least the minimum expected components on every PR.



Tool usage is healthy

tool_efficiency 0.89 means the agent rarely repeats itself. tool_coverage 0.86 means it usually calls the right tools — only missing some on simpler PRs where fewer tools are needed.



Risk detection needs improvement

risk_quality 0.66 and groundedness 0.72 — the agent finds roughly 2/3 of the risks a human reviewer would flag. It misses subtle edge cases and sometimes introduces weakly-grounded claims.

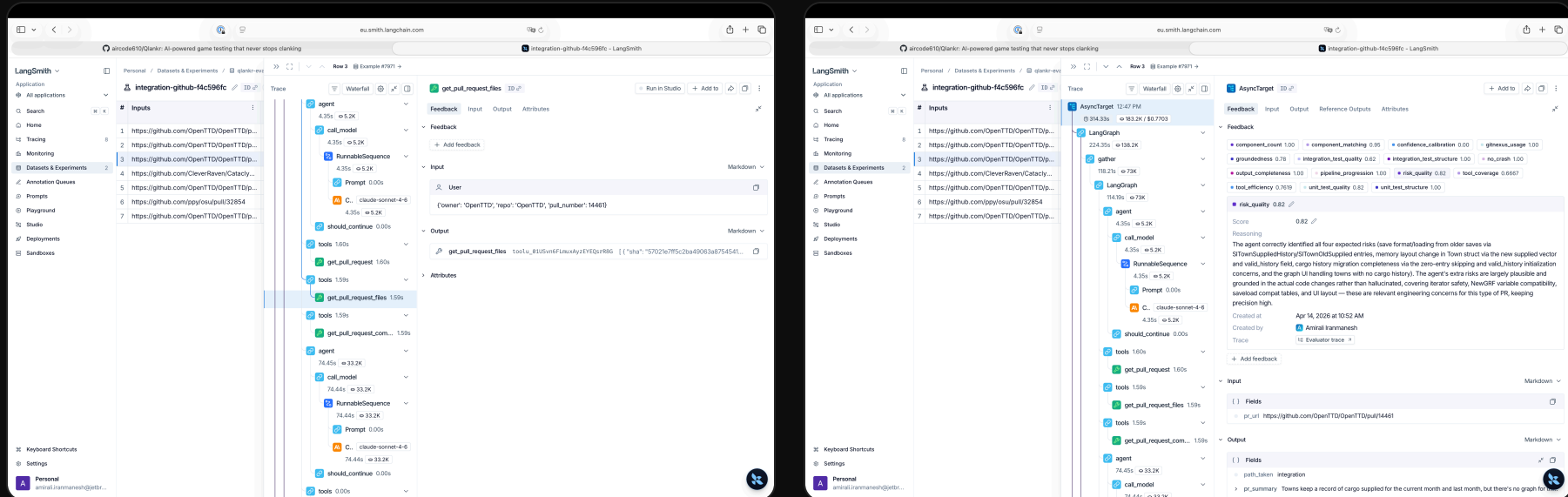


Integration tests are the weakest point

integration_test_structure 0.29 and quality 0.18 — the agent often exhausts its tool budget on research without calling submit. Confidence calibration 0.18 shows it's also rating non-indexed repos too highly.

LANGSMITH IN ACTION

Every run is **fully traceable**



Inspect **every tool call** in a single run — what arguments were passed, what was returned, where it went wrong.

THE LOOP

Change → Test → Compare

1 Change a prompt or model

Edit the gather/integration/e2e system prompt, swap model, adjust budget limit.

2 Run evals

```
python -m evals.run_evals --suite integration --dataset github
```

3 Compare in LangSmith

Side-by-side score diff for every metric — see exactly which evaluators improved and which regressed.

SUMMARY

From "looks good" to reproducible scores



The problem

AI agents are non-deterministic, multi-step, and silent when they regress. Manual review doesn't scale.



Golden dataset

8 real PRs, expert-labeled with expected components, risks, and required tools.



3 evaluator layers

Pipeline health (free), behaviour checks (free), LLM quality judges — 15 metrics total.



What we found

Pipeline: solid. Risk detection: decent. Integration tests + confidence calibration: next targets.